# Cuprinsul cursului de azi

1. Căutare informată
   - Căutare Greedy (Greedy Best-First Search)
   - Euristici admisibile
   - Căutare A* (A star Best-First Search)

# Euristică admisibilă

- O euristică $h$ este *admisibilă* (optimistă) dacă:

$$0 \leq h(n) \leq h^*(n)$$

unde $h^*(n)$ este costul real către cel mai apropiat nod scop

- **O euristică admisibilă nu supraestimează niciodată** distanța până la scop

- f(n) = g(n) + h(n) **nu supraestimează** costul real al unui drum care trece prin nodul n, atunci când h(n) este admisibilă

# Algoritmul de căutare A*

- Cel mai cunoscut algoritm de tip **Best-First Search**
- Evaluează nodurile *n* folosind:
  - *g(n)* – costul exact de la starea inițială la *n*
  - *h(n)* – estimarea costului de la *n* la scop
- Funcția de evaluare: $f(n) = g(n)+h(n)$
- *f(n)* = estimarea costului total al unui drum optim care trece prin *n*
- Selectează nodul din **OPEN LIST** cu **valoarea minimă a lui *f(n)***
- Performanța depinde de calitatea funcției euristice *h*

# Căutare A* folosind TREE-SEARCH

**funcția Astar-TREE-SEARCH** (**problema**, **h**) **returnează** o soluție sau eșec

inițializează **frontiera (Open List)** folosind **starea inițială a problemei**

**ciclează**

**dacă frontiera** este vidă **atunci returnează** eșec

**nod = pop_min_f(Open List)**

**dacă nodul** conține o **stare scop**

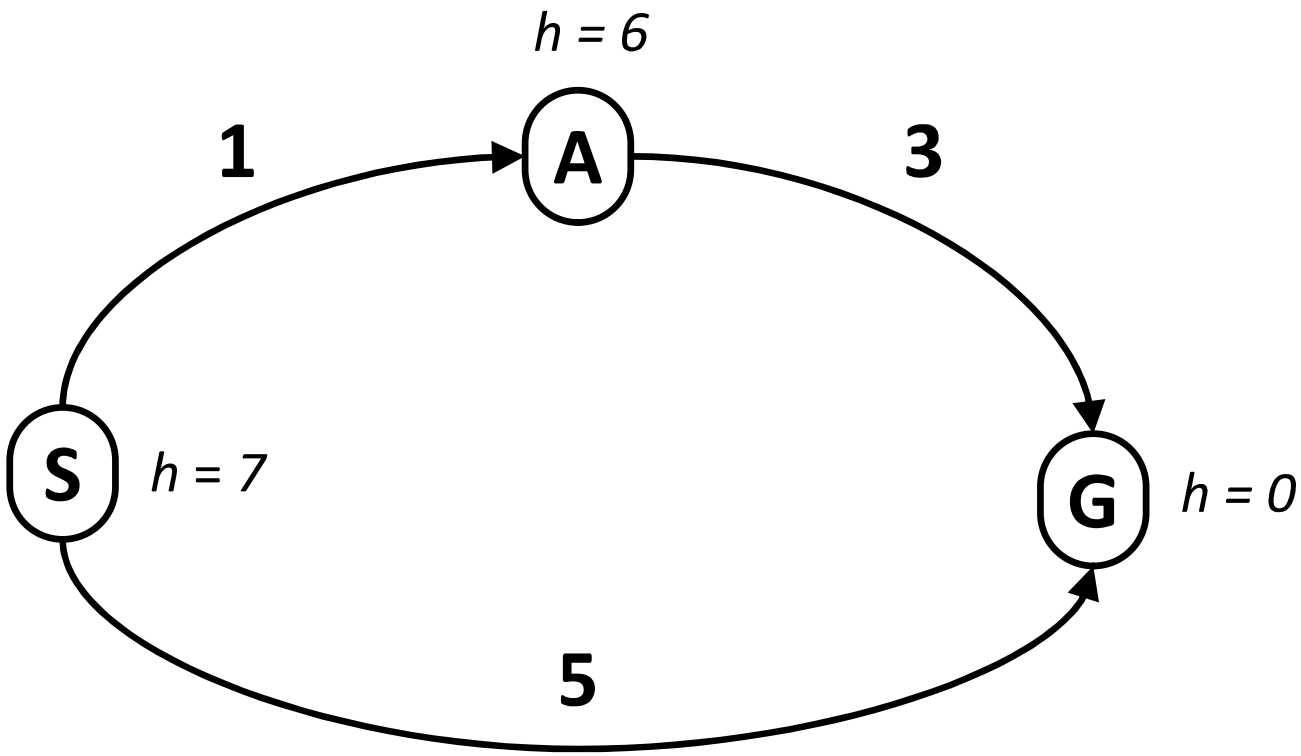
**atunci returnează** soluția corespunzătoare

expandează nodul ales, adăugând nodurile succesor generate în **frontieră**

**Frontiera (OPEN LIST) pentru A*:**

- structură de date cu noduri generate dar neexpandate
- implementată ca o **coadă de priorități**
- ordonată crescător după $f(n) = g(n) + h(n)$
- la fiecare pas se extrage nodul cu **valoarea minimă a lui *f(n)***

# Este algoritmul de căutare A* optimal?




- Găsește soluția S-G de cost 5 deși soluția optimă este S-A-G de cost 4.
- Avem nevoie de estimări (date de euristici) care să fie mai mici decât costurile reale (euristici optimiste)
- Optimalitate (în cazul Tree-Search) condiționată de euristici cu o asemenea caracteristică: A* este optim dacă avem euristici optimiste

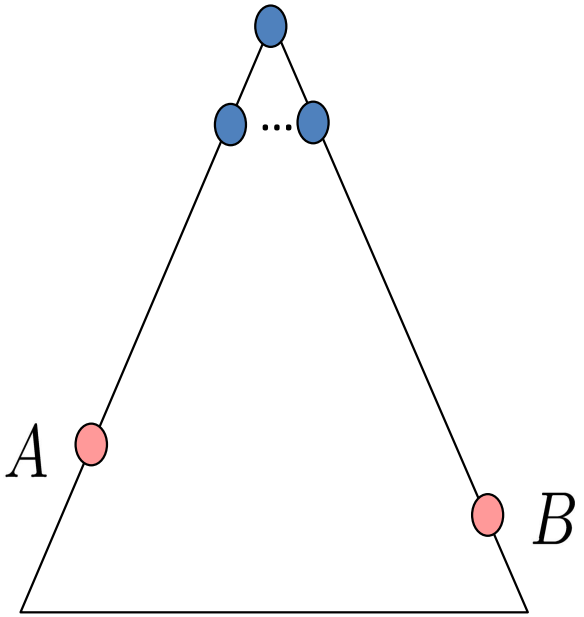
# Optimalitatea algoritmului de căutare A* folosind Tree-Search

Presupunem:

- A este un nod-scop optim global (nu există alt nod-scop cu cost mai mic)
- B este un nod-scop suboptimal (are cost mai mare decât soluția optimă)
- h este euristică admisibilă

Afirmație:

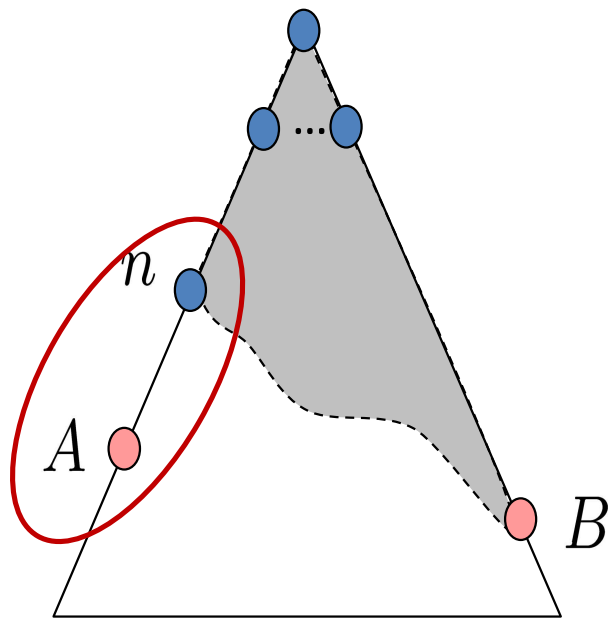
- A va fi explorat de algoritmul de căutare A* înaintea lui B



# Optimalitatea algoritmului de căutare A* folosind Tree-Search

Demonstrație:

- presupunem că B este pe frontieră
- un nod ascendent *n* al lui A (strămoș) este de asemenea pe frontieră (poate chiar nodul A!)
- afirmație: *n* va fi explorat înaintea nodului B
  1. f(n) ≤ f(A)




$f(n) = g(n) + h(n)$ **definiția costului f**

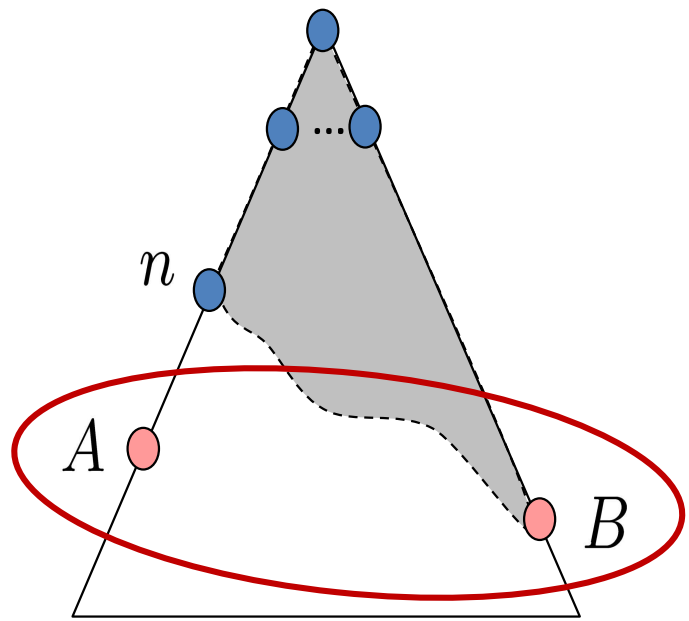
$f(n) \leq g(A)$ **h subestimează costul până la A**

$g(A) = f(A)$ **h(A) = 0, A e nod-scop**

# Optimalitatea algoritmului de căutare A* folosind Tree-Search

Demonstrație:

- presupunem că B este pe frontieră
- un nod ascendent *n* al lui A (strămoș) este de asemenea pe frontieră (poate chiar nodul A!)
- afirmație: *n* va fi explorat înaintea nodului B
  1. f(n) ≤ f(A)
  2. f(A) < f(B)




$g(A) < g(B)$ **B este nod suboptimal**

$f(A) < f(B)$ **h(A) = 0, h(B) = 0**

h e euristică

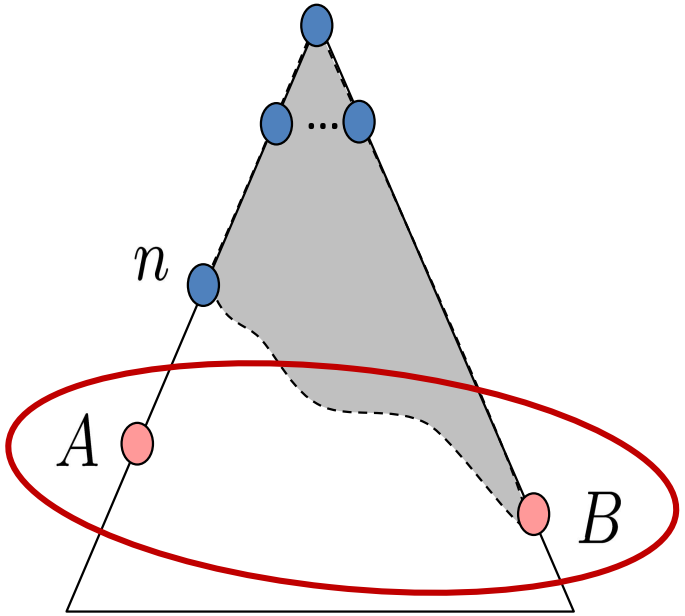
$$f(A) = g(A) + h(A) = g(A)$$

$$f(B) = g(B) + h(B) = g(B)$$

# Optimalitatea algoritmului de căutare A* folosind Tree-Search

Demonstrație:

- presupunem că B este pe frontieră
- un nod ascendent *n* al lui A (strămoș) este de asemenea pe frontieră (poate chiar nodul A!)
- afirmație: *n* va fi explorat înaintea nodului B
  1. f(n) ≤ f(A)
  2. f(A) < f(B)
  3. n este explorat înaintea lui B

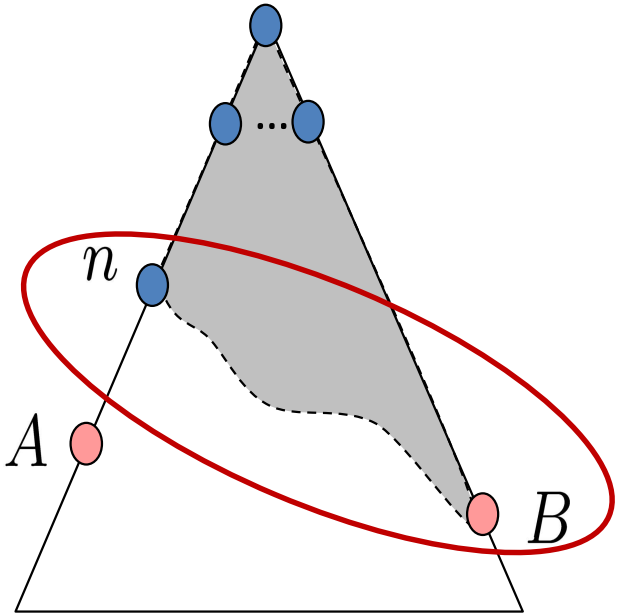



$$f(n) \leq f(A) < f(B)$$

# Optimalitatea algoritmului de căutare A* folosind Tree-Search

Demonstrație:

- presupunem că B este pe frontieră
- un nod ascendent *n* al lui A (strămoș) este de asemenea pe frontieră (poate chiar nodul A!)
- afirmație: *n* va fi explorat înaintea nodului B
  1. $f(n) \leq f(A)$
  2. $f(A) < f(B)$
  3. *n* este explorat înaintea lui B
- toți ascendenții lui A sunt explorați înaintea lui B
- A este explorat înaintea lui B
- algoritmul A* este optimal



# Căutare A* folosind TREE-SEARCH

**funcția Astar-TREE-SEARCH** (**problema**, **h**) **returnează** o soluție sau eșec

inițializează **frontiera (Open List)** folosind **starea inițială a problemei**

**ciclează**

**dacă frontiera** este vidă **atunci returnează** eșec

**nod = pop_min_f(Open List)**

**dacă nodul** conține o **stare scop**

**atunci returnează** soluția corespunzătoare

expandează nodul ales, adăugând nodurile succesor generate în **frontieră**

**Frontiera (OPEN LIST) pentru A*:**

- structură de date cu noduri generate dar neexpandate
- implementată ca o **coadă de priorități**
- ordonată crescător după $f(n) = g(n) + h(n)$
- la fiecare pas se extrage nodul cu **valoarea minimă a lui *f(n)***

**Optimal dacă h este admisibilă**

# Căutare A* folosind GRAPH-SEARCH

**funcția Astar--GRAPH-SEARCH** (**problema**, **h**) **returnează** o soluție sau eșec

inițializează **frontiera (Open List)** folosind **starea inițială a problemei**
inițializează mulțimea (Closed List) de stări explorate (vizitate) cu mulțimea vidă
**ciclează**
**dacă frontiera** este vidă **atunci returnează** eșec
**nod = pop_min_f(Open List)**
**dacă nodul** conține o **stare scop**
**atunci returnează** soluția corespunzătoare
dacă starea nodului curent este în Closed List
**atunci continuă**
adaugă starea nodului curent la Closed List
expandează nodul ales, adăugând nodurile succesor generate în **frontieră**

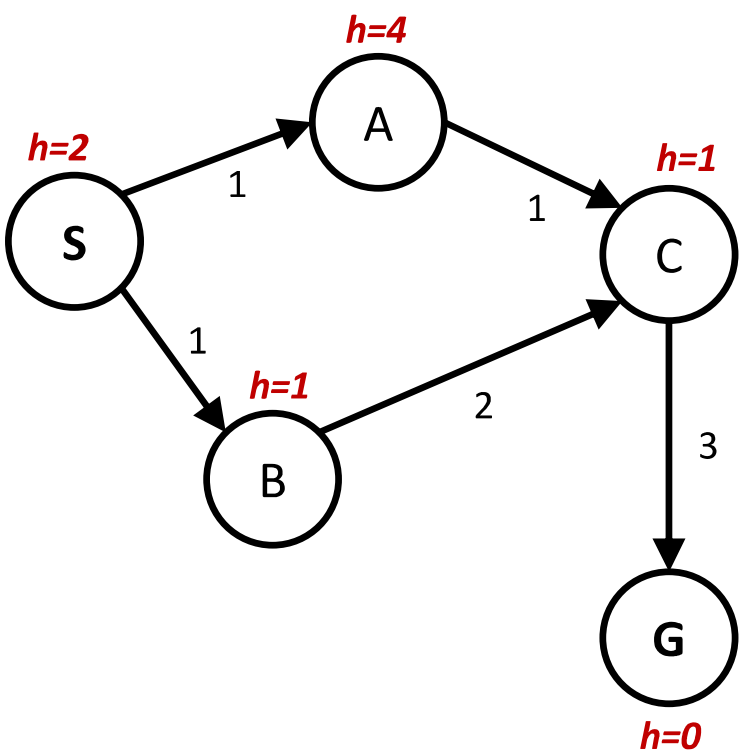
**Frontiera (OPEN LIST) pentru A*:**

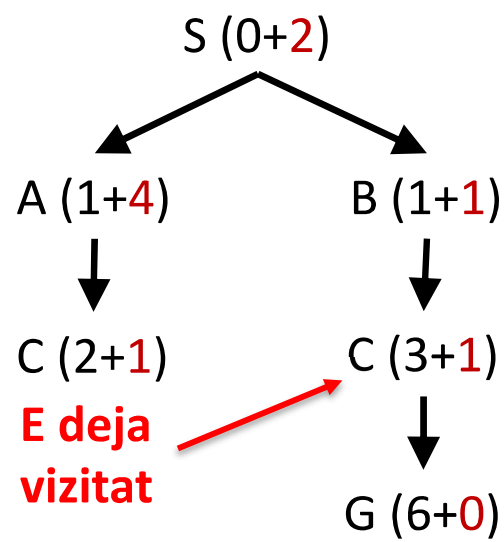
- structură de date cu noduri generate dar neexpandate
- implementată ca o **coadă de priorități**
- ordonată crescător după $f(n) = g(n) + h(n)$
- la fiecare pas se extrage nodul cu **valoarea minimă a lui *f(n)***

# Algoritmul A* folosind Graph-Search

- Poate returna soluții suboptimale chiar dacă folosim o euristică admisibilă
- A* folosind Graph-Search poate elimina un drum optim către o stare deja vizitată dacă acesta nu este primul generat *(un drum ulterior poate fi mai bun)*
- Problema apare deoarece nu reexpandează stările deja explorate
- Putem evita acest comportament impunând o condiție asupra euristicii (să fie consistentă)

# Exemplu cu algoritmul A* folosind Graph-Search




**Nu obțin soluția optimă**
**Obțin soluția S – B – C – G cu costul 6**
**Dar soluția S – A – C – G are costul 5**

# Euristică consistentă

- O euristică este consistentă (monotonă) dacă:

$$| h(a) - h(b) | \leq c(a,b)$$

unde:

- *h(n)* = estimarea costului de la nodul n până la scop
- *c(a,b)* = costul real al tranziției de la a la b

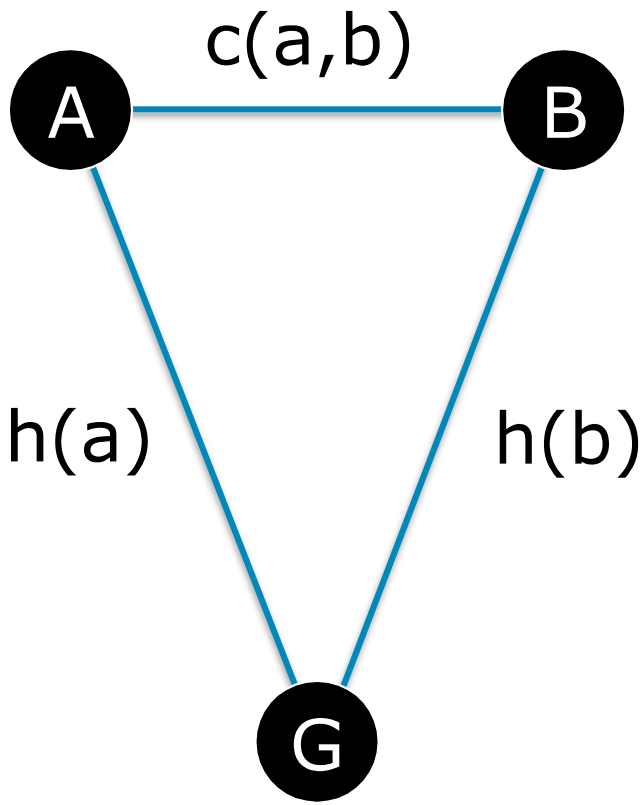
**Intuiție**

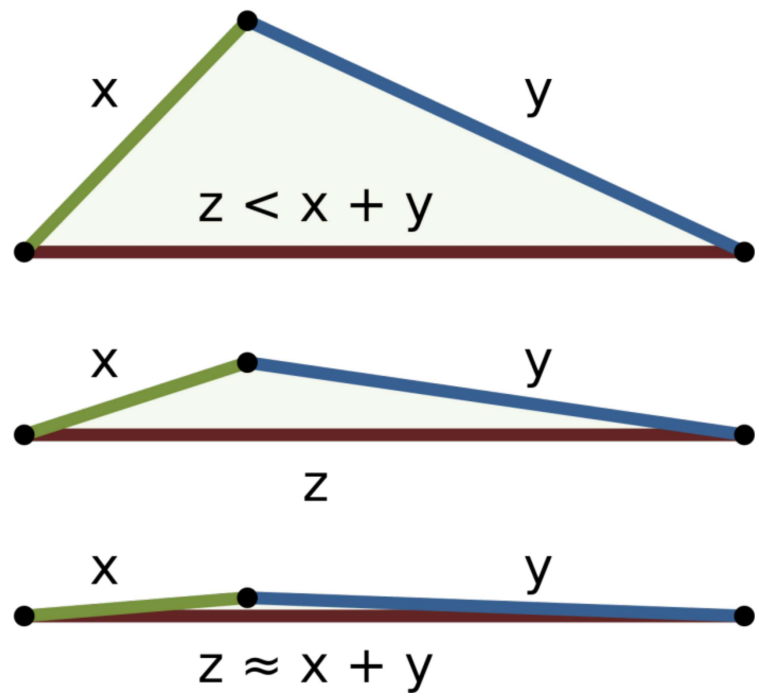
- Diferența dintre valorile euristice a două stări vecine nu poate depăși costul real dintre ele

- Euristica h „nu sare” prea mult între stări

# Euristică consistentă

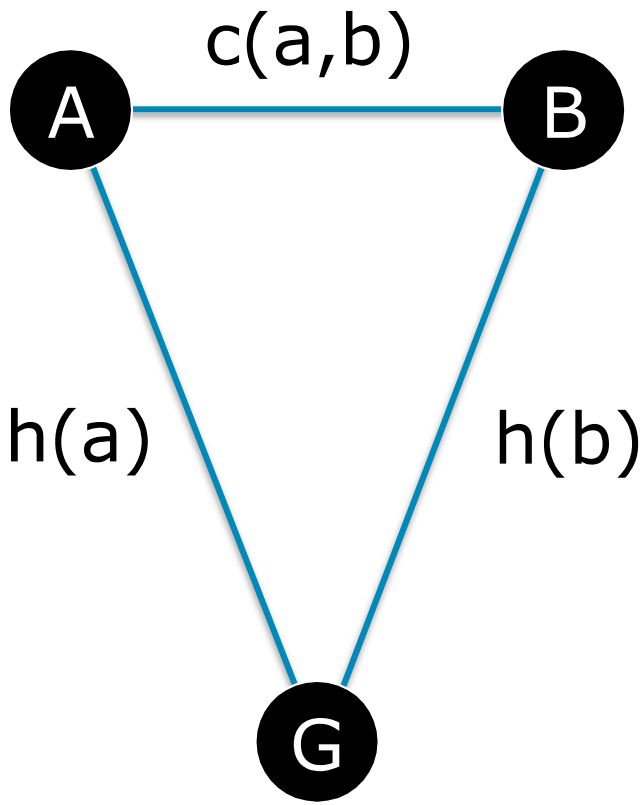
$$| h(a) - h(b) | \leq c(a,b)$$

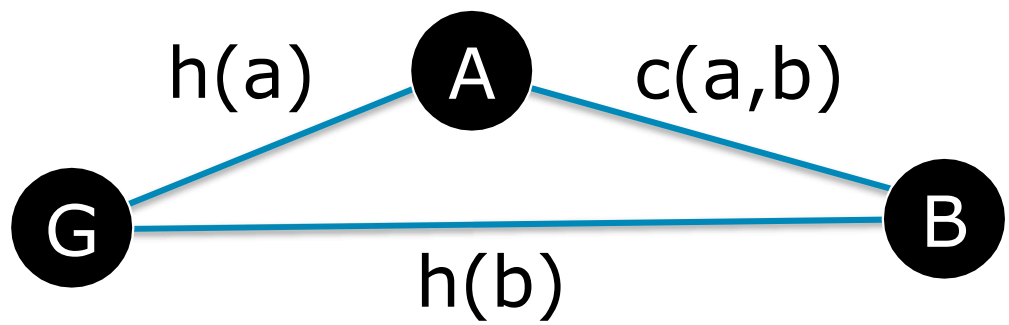


# Euristică consistentă

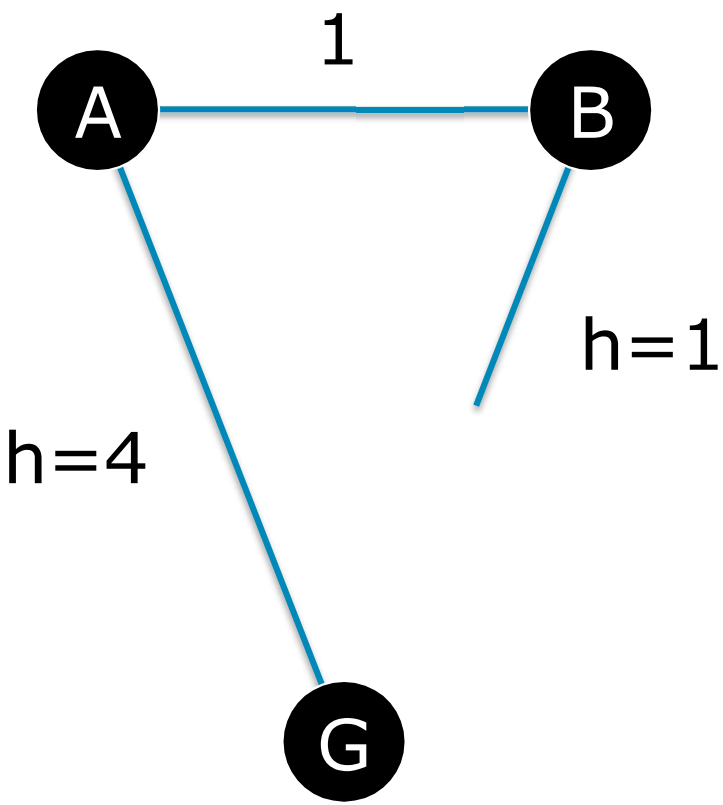
$$| h(a) - h(b) | \leq c(a,b)$$



# Euristică consistentă

$$| h(a) - h(b) | \leq c(a,b)$$




Inconsistentă

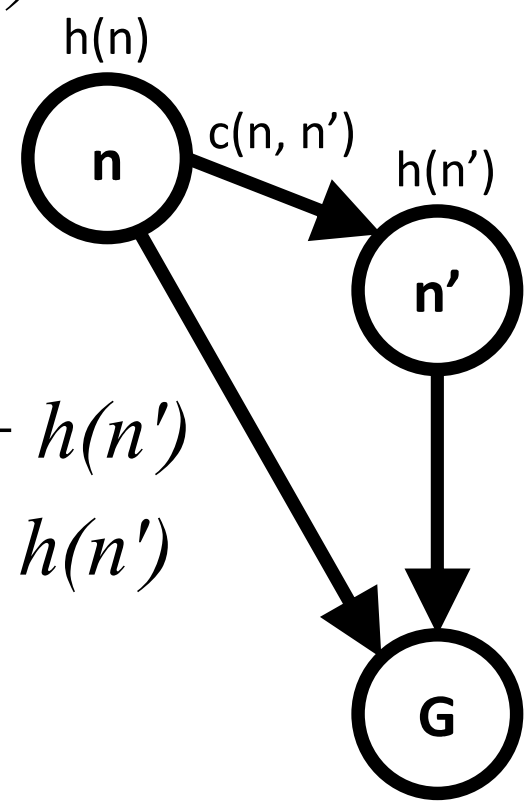
# Proprietăți ale euristicii consistente

- Orice euristică consistentă este și admisibilă
- În practică, este dificil să construim o euristică admisibilă care să nu fie și consistentă
- Dacă $h(n)$ este consistentă, atunci valorile $f(n) = g(n) + h(n)$ sunt monotone nedescrescătoare de-a lungul oricărui drum
- Nodurile sunt expandate de către căutarea A* folosind Graph-Search în ordine crescătoare a lui $f(n)$
- Primul nod scop selectat pentru expandare este optimal (pentru că celelalte noduri au cost cel puțin la fel de mare)

# Proprietăți ale euristicii consistente

- Dacă $h(n)$ este consistentă, atunci valorile $f(n) = g(n) + h(n)$ sunt monotone nedescrescătoare de-a lungul oricărui drum

- Condiția de consistență: $h(n) \leq c(n,n') + h(n')$

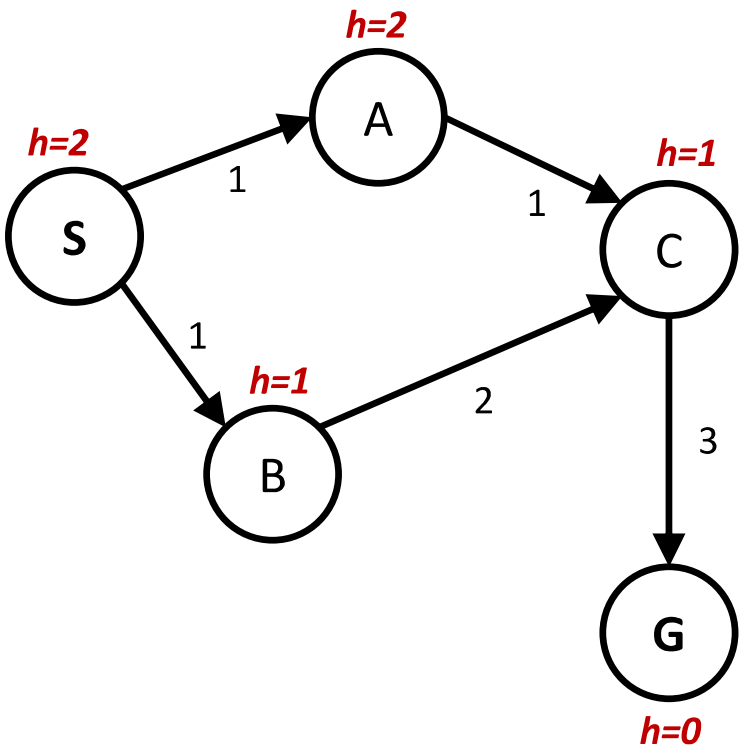


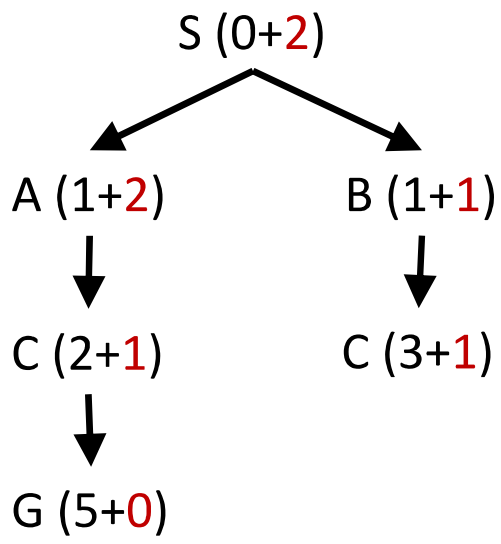

- Pe un drum n → n′ avem:
  - $f(n) = g(n) + h(n)$
  - $f(n') = g(n') + h(n') = g(n) + c(n,n') + h(n')$
  - din consistență avem $h(n) \leq c(n, n') + h(n')$

  $\Rightarrow g(n) + h(n) \leq g(n) + c(n,n') + h(n')$

  $\Rightarrow f(n) \leq f(n')$

# Exemplu cu algoritmul A* folosind Graph-Search




**Obțin soluția optimă**
**S – A – C – G are costul 5**

# Căutare A* folosind GRAPH-SEARCH

**funcția Astar--GRAPH-SEARCH** (**problema**, **h**) **returnează** o soluție sau eșec

inițializează **frontiera (Open List)** folosind **starea inițială a problemei**
inițializează mulțimea (Closed List) de stări explorate (vizitate) cu mulțimea vidă
**ciclează**
  **dacă frontiera** este vidă **atunci returnează** eșec
  **nod = pop_min_f(Open List)**
  **dacă nodul** conține o **stare scop**
    **atunci returnează** soluția corespunzătoare
  dacă starea nodului curent este în Closed List
    **atunci continuă**
  adaugă starea nodului curent la Closed List
  expandează nodul ales, adăugând nodurile succesor generate în **frontieră**

**Frontiera (OPEN LIST) pentru A*:**

- structură de date cu noduri generate dar neexpandate
- implementată ca o **coadă de priorități**
- ordonată crescător după $f(n) = g(n) + h(n)$
- la fiecare pas se extrage nodul cu **valoarea minimă a lui *f(n)***

**Optimal dacă h este consistentă**

# Implementare A* folosind GRAPH-SEARCH

```
Function AStar(problem, h)
  closed = {}
  open = PQ(f=g+h, Node(problem.init_state))
  while (true)
    if (open.empty) return fail
    node = pop_min_f(open)
    if (node.state is goal) return solution
    if (node.state in closed) continue
    closed.add(node.state)
    for c in Expand(node, problem)
      if (c.state in closed) continue

      open.add(c)
```

**Dacă o stare a unui nod succesor a fost deja explorată (se află în CLOSED), nu va mai fi procesată din nou.**

**Reduce memoria utilizată de OPEN List**

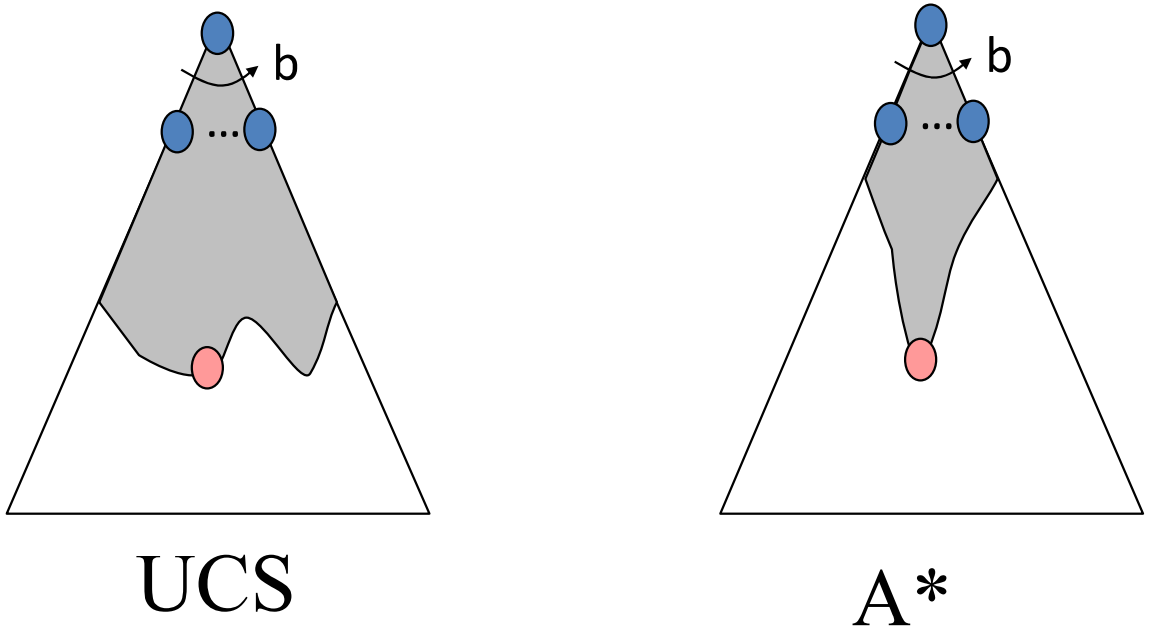
# Implementare A* folosind GRAPH-SEARCH

```
Function AStar(problem, h)
  closed = {}
  open = PQ(f=g+h, Node(problem.init_state))
  while (true)
    if (open.empty) return fail
    node = pop_min_f(open)
    if (node.state is goal) return solution
    if (node.state in closed) continue
    closed.add(node.state)
    for c in Expand(node, problem)
      if (c.state in closed) continue
      if (node n in open with c.state and n.g <= c.g) continue
      open.add(c)
```

**Dacă există deja un drum mai bun către această stare în OPEN, ignorăm nodul.**

**Poate fi foarte lent în practică**

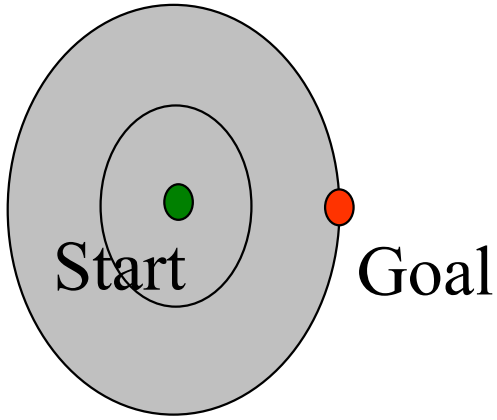
# Proprietățile algoritmului A*



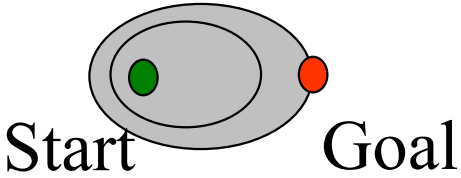

Căutare uniformă pe bază de cost explorează drumuri de cost minim, indiferent de cât de îndepărtate sunt de soluția-scop. Algoritmul A* beneficiază de euristica *h* care restricționează căutarea.

# UCS vs A* - explorarea arborelui de căutare

- Căutarea uniformă pe bază de cost (UCS) explorează soluții uniform în toate direcțiile.



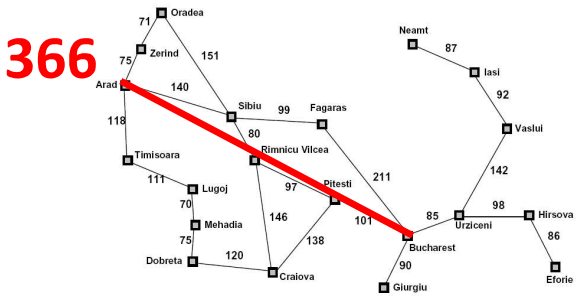

- Algoritmul A* explorează soluții în mare parte în direcția soluției, totuși păstrează și soluții în alte direcții (pentru păstrarea optimalității).

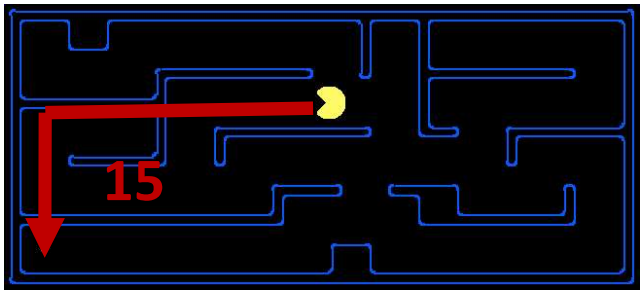


# Crearea de euristici admisibile

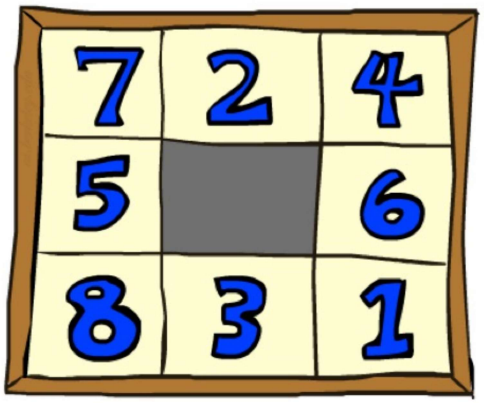
- Un ingredient fundamental în rezolvarea optimă a problemelor de căutare este găsirea unor euristici admisibile

- Deseori, euristicile admisibile sunt soluții pentru *probleme relaxate (obținute din problema inițială prin relaxarea unor condiții),* când sunt disponibile noi acțiuni

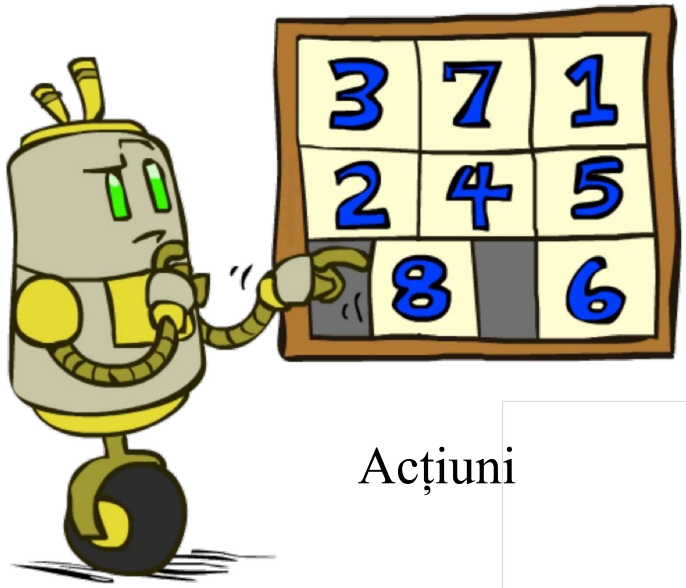


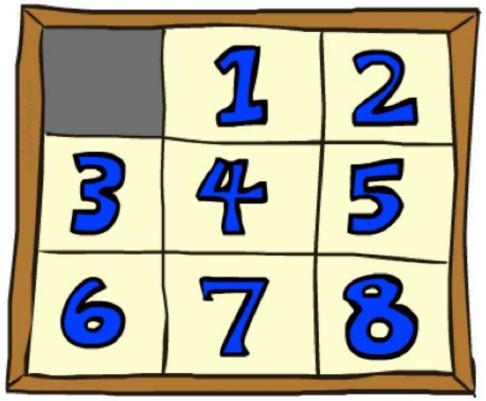
# Exemplu – 8 Puzzle




Starea inițială



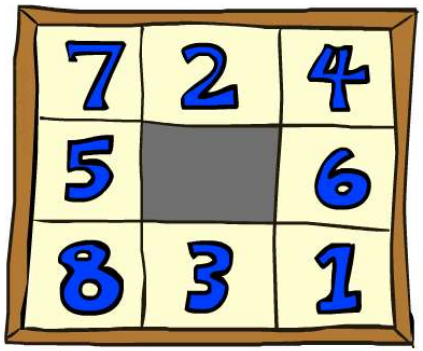

Acțiuni



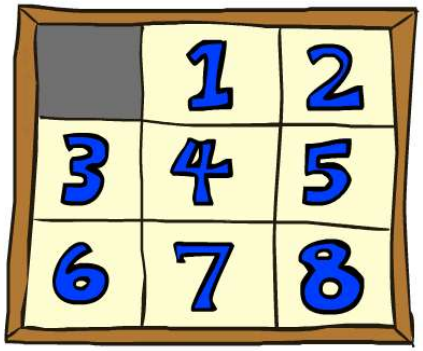

Starea finală

# Exemplu – 8 Puzzle

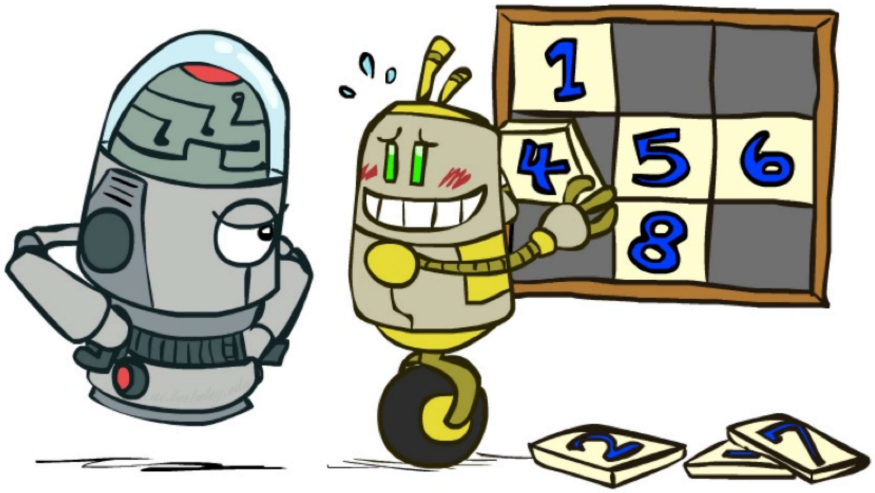
- Euristică: numărul de piese puse greșit
- Este euristică admisibilă?
- h(stare inițială) = 8
- Aceasta este o euristică pentru o *problema relaxată*




Starea inițială



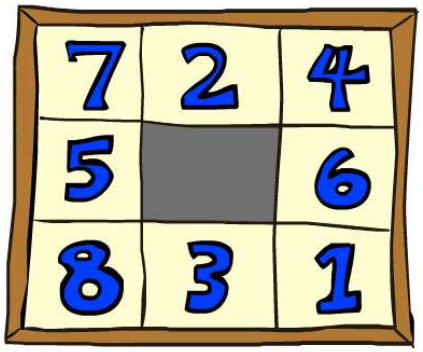

Starea finală



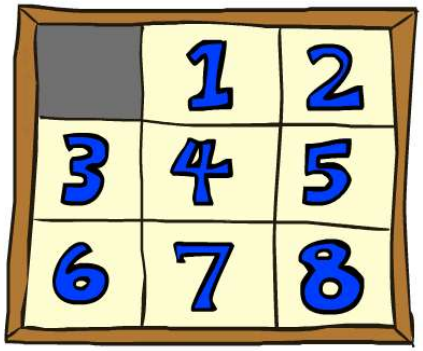

| | Numărul de noduri explorate când soluția optima are... | | |
|---|---|---|---|
| | ...4 pași | ...8 pași | ...12 pași |
| UCS | 112 | 6,300 | $3.6 \times 10^6$ |
| A* + #TILES | 13 | 39 | 227 |

# Exemplu – 8 Puzzle

- Dacă considerăm problema 8-puzzle modificată astfel încât fiecare piesă poate fi glisată stânga/dreapta/sus/jos ignorând celelalte piese?




Starea inițială




Starea finală

- Distanța *Manhattan*
- Este euristică admisibilă?
- h(stare inițială) = 3 + 1 + 2 + … = 18

| | Numărul de noduri explorate când soluția optima are… | | |
|---|---|---|---|
| | …4 pași | …8 pași | …12 pași |
| A* + #TILES | 13 | 39 | 227 |
| A* + MANHATTAN | 12 | 25 | 73 |